

```
manchodu@Srini:~/OSLAB$ valgrind --leak-check=full --show-leak-kinds=all ./t
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
manchodu@Srini:~/OSLAB$ |
```

```
manchodu@Srini:~/OSLAB$ cat m1.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    printf("==== malloc(), calloc(), realloc(), free() ==== \n\n");

    /* malloc() */
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
        a[i] = (i + 1) * 10;

    printf("Memory allocated using malloc(): ");
    for (i = 0; i < 5; i++)
        printf("%d ", a[i]);
    printf("\n");

    /* calloc() */
    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("Memory allocated using calloc(): ");
    for (i = 0; i < 5; i++)
        printf("%d ", b[i]);
    printf("\n");

    /* realloc() */
    int *temp = (int *)realloc(a, 10 * sizeof(int));

    if (temp == NULL)
    {
        printf("realloc failed\n");
    }
}
```

```

for (i = 0; i < 5; i++)
    printf("%d ", a[i]);
printf("\n");

/* calloc() */
int *b = (int *)calloc(5, sizeof(int));

if (b == NULL)
{
    printf("calloc failed\n");
    free(a);
    return 1;
}

printf("Memory allocated using calloc(): ");
for (i = 0; i < 5; i++)
    printf("%d ", b[i]);
printf("\n");

/* realloc() */
int *temp = (int *)realloc(a, 10 * sizeof(int));

if (temp == NULL)
{
    printf("realloc failed\n");
    free(a);
    free(b);
    return 1;
}

a = temp;

for (i = 5; i < 10; i++)
    a[i] = (i + 1) * 10;

printf("Memory after realloc(): ");
for (i = 0; i < 10; i++)
    printf("%d ", a[i]);
printf("\n");

/* free() */
free(a);
free(b);

printf("\nAll dynamically allocated memory has been freed.\n");

return 0;

```

anubhav@Srinivas:~/OS LAB\$

```
manchodu@Srini:~/OSLAB$ nano m2.c
manchodu@Srini:~/OSLAB$ gcc m2 -o m2.c
/usr/bin/ld: cannot find m2: No such file or directory
collect2: error: ld returned 1 exit status
manchodu@Srini:~/OSLAB$ gcc m2.c -o m2
cc1: fatal error: m2.c: No such file or directory
compilation terminated.
manchodu@Srini:~/OSLAB$ gcc -o m2 m2.c
cc1: fatal error: m2.c: No such file or directory
compilation terminated.
manchodu@Srini:~/OSLAB$ nano m2.c
manchodu@Srini:~/OSLAB$ gcc -o m2 m2.c
manchodu@Srini:~/OSLAB$ ./m2.c
-bash: ./m2.c: Permission denied
manchodu@Srini:~/OSLAB$ |
```

```

{
    /* Child process */
    printf("==== CHILD PROCESS =====\n");
    printf("Child PID: %d\n", getpid());
    printf("Parent PID: %d\n", getppid());

    printf("Before modification:\n");
    printf("data[0] = %d\n", data[0]);
    printf("Address of data: %p\n", (void *)data);

    sleep(2);

    /* Modify memory in child */
    data[0] = 9999;

    printf("\nAfter modification in child:\n");
    printf("data[0] = %d\n", data[0]);
    printf("Address of data: %p\n", (void *)data);

    printf("\nChild modified its copy of the memory.\n");
    printf("Copy-on-Write has occurred.\n");

    free(data);
    exit(0);
}
else
{
    /* Parent process */
    printf("==== PARENT PROCESS =====\n");
    printf("Parent PID: %d\n", getpid());

    sleep(1);

    printf("Before child modification:\n");
    printf("data[0] = %d\n", data[0]);
    printf("Address of data: %p\n", (void *)data);

    wait(NULL);

    printf("\nAfter child modification:\n");
    printf("data[0] = %d\n", data[0]);

    printf("\nParent value remains unchanged.\n");
    printf("This demonstrates Copy-on-Write behavior.\n");

    free(data);
}

return 0;

```

```

#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main()
{
    int i;

    /* Allocate memory before fork */
    int *data = (int *)malloc(SIZE * sizeof(int));

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (i = 0; i < SIZE; i++)
        data[i] = i;

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated: %d integers\n", SIZE);
    printf("Address of data: %p\n\n", (void *)data);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
    {
        /* Child process */
        printf("==== CHILD PROCESS =====\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Before modification:\n");
        printf("data[0] = %d\n", data[0]);
        printf("Address of data: %p\n", (void *)data);

        sleep(2);
    }
}

```